

Information Processing 1 — Week 14 exercises

Projects can be submitted the same week or the following week with no penalty.

New projects will be added each week. Each project is worth a certain number of points. You can score up to 50 points in total from all the projects you complete.

When you finish a project, **immediately** ask an instructor to grade it. If you are stuck or have any questions then **raise your hand** and an instructor will help you.

1 Database management [7 points]

Databases are a crucial data organisation tool for experimental scientific research and for engineering design, manufacturing, and automation. Databases are also used in project management and revision control systems for software engineering. (If you have ever used git, or Subversion, or Mercurial, they are effectively databases optimise for storing program source code.) Content management systems for web sites and web services, including Wikis and blogging platforms such as Wordpress as well as Google or Microsoft online office applications, all make heavy use of databases.

Some popular database management systems found on Linux, MacOS and Windows are MySQL, MariaDB, and PostgreSQL. Their interactive management interfaces are large and complex, but in some ways also similar to the small interface that you will develop in this project. If you ever become admin of a lab computer running communication or data logging services, you will probably have to manipulate a database using an interface similar the one you build here.

1.1 Task

- read a command of the form '**name** **action** [*arguments*...]' where **name** is the name of a database file and **action** is one of **set**, **get**, **del**, **keys**, or **values** followed by zero or more *arguments* (the number depends on which action is being performed)
- if the action is '**set**', open the database for writing, write the supplied value at the given key, then close the database
- if the action is '**get**', open the database for reading, read the value at the given key and print it, then close the database
- if the action is '**del**', open the database for reading, delete the specified key, then close the database
- if the action is '**keys**', open the database for reading, print the valid keys in the database, then close the database
- if the action is '**values**', open the database for reading, read the valid keys in the database and then print each key with its value, then close the database.

1.2 Example session

You can write this program to either read the database name, action, and arguments from the command line or from a 'shell' loop inside your program that uses `input()` to read the same information.

If you write the program to take information from the command line, a session might look like this.

```

$ python3 database.py japan set population 125470000
$ python3 database.py japan set capital Tokyo
$ python3 database.py japan set language Japanese
$ python3 database.py france set population 67413000
$ python3 database.py france set capital Paris
$ python3 database.py france set language French
$ python3 database.py japan get capital
Tokyo
$ python3 database.py japan get language
Japanese
$ python3 database.py france get capital
Paris
$ python3 database.py japan keys
capital population language
$ python3 database.py japan values
capital Tokyo
population 125470000
language Japanese
$ python3 database.py japan del capital
$ python3 database.py japan keys
population language
$ python3 database.py japan
usage:
  database.py database-name set key value
  database.py database-name get key
  database.py database-name del key
  database.py database-name keys
  database.py database-name values
$ █

```

If you use `input()` to implement your own 'shell' then a session might look like this.

```

$ python3 database.py
? japan set population 125470000
? japan set capital Tokyo
? japan set language Japanese
? france set population 67413000
? france set capital Paris
? france set language French
? japan get capital
Tokyo
? japan get language
Japanese
? france get capital
Paris
? japan keys
capital population language

```

```

? japan values
capital Tokyo
population 125470000
language Japanese
? japan del capital
? japan keys
population language
? japan
usage:
  database-name set key value
  database-name get key
  database-name del key
  database-name keys
  database-name values
$ █

```

1.3 Program structure

You could separate the program into two parts: (1) processing the user's input, and (2) performing the task specified by the input. The interface to part (2) (the part that performs the database operations) can look the same in both cases.

1.4 Database operations

Use the `dbm` module to help implement database operations. You might implement the operations as a function `perform(database, command, arguments)` where:

- `database` is a string giving the name of the database,
- `command` is a string giving the command to perform, and
- `arguments` is a list of strings containing the arguments required by the `command`.

The body of `perform` can be a sequence of `if...elif...elif...else` statements that test which of the five possible commands has been given and then for each possibility they implement the database `open`, manipulation, and `close` operations.

1.5 Input from the command line

Constructing the arguments for `perform` is easy. Everything is already in `sys.argv` and all that is needed is to split its contents up appropriately. For example, you might do something like this:

```

import sys
# sys.argv is: ["database.py", dbName, command, key, value]
database = sys.argv[1] # dbName
command = sys.argv[2] # command
arguments = sys.argv[3:] # arguments
perform(database, command, arguments)

```

1.6 Input from a simulated shell using `input()`

The `input()` function returns a single string. You strip off any whitespace from the beginning and end, then split the string into words. The rest then looks just like the command line version.

```
while True:
    argv = input("? ").strip().split()
    # argv is: [dbName, command, key, value]
    database = argv[0]
    command = argv[1]
    arguments = argv[2:]
    perform(database, command, arguments)
```

1.7 Validation of input

Each operation requires at least the name of a database and a command. The `keys` and `values` commands need no additional arguments, `get` and `del` need one (a key) and `set` needs two (a key and a value). Validate the number of arguments in the appropriate places. If the wrong number of arguments is given, print a message explaining the usage of each command and then use `exit()` to terminate the program. Do the same if an unknown command (or no command and/or database name) is given.

The `get` and `del` commands will fail if they try to access or delete a non-existent key. Verify that the key exists in the database before attempting to read or delete it. (There is no need to print any kind of error message if the key is missing.)

1.8 Converting bytes to a string

You will notice that all your keys and values are printed as `b'something'` which indicates they are raw bytes and not strings. To convert a bytes value `b` to a string you can use `b.decode()`. Use this to print the output of the `get`, `keys`, and `values` commands as strings.

2 Monte Carlo simulation to find π [5 points]

Monte Carlo simulation uses massive random sampling to obtain numerical results based on theoretical models of physical systems. Parameters that are part of a theoretical model of a system can be given a concrete value by performing many random trials, measuring the outcomes, and then relating the result back to the theoretical model. Monte Carlo simulation is an important technique in many areas of engineering and experimental scientific research.

Monte Carlo simulation is named after the town of Monte Carlo in Monaco, located between France and Italy, that is famous for its gambling casino.

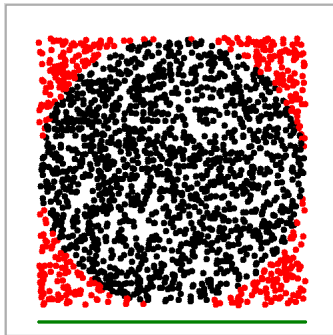
2.1 Task

- generate a random pair of values x and y in the range $[-1, 1]$

- determine whether the point (x, y) lies within the unit circle¹
 - if it does, increment a **tally** of points inside the circle
- repeat the experiment for **count** times
- use the ratio of **tally** to **count** to estimate the value of π , then print it.

To make it more interesting to look at, use the **turtle** module to draw each point as the simulation progresses. Draw the point in black if it is inside the circle or in red if it is outside.

2.2 Example session



Pi is 3.14124

2.3 Program structure

You will need to import the **turtle** and **random** modules.

The **random.random()** function gives you a random floating point number in the range $[0.0, 1.0]$.

To simplify the drawing you can use a fixed-size window. In the example above I used a window of 500 by 500 pixels which you can obtain by using **setup(500, 500)** at the start of your program (assuming you have done **from turtle import ***).

To leave a little space around the circle I drew the points scaled by 200 which placed them at least 50 pixels from the edge of the window. (Recall that the origin is at the centre of the window.)

To make the graphics faster you should turn off automatic updates, hide the turtle, and lift the pen up at the start of the program.

```
setup(500, 500)
tracer(0, 0)
hideturtle()
up()
```

Each point lies somewhere inside a square with corners at $(-1, -1)$ and $(1, 1)$. To check whether it is inside the unit circle you could use Pythagoras' formula to obtain the distance from the origin to the point and check whether that distance is less than **1.0**.

The **'dot(s, c)'** function draws a dot of radius **s** pixels using the colour named by the string **c**. When drawing each 1000th point, use black if it is inside the unit circle or red if not. Assuming you have set the variable **inside** to **True** if the point lies within the unit circle then you could draw it using something like this:

¹The unit circle is the circle with radius 1 centred on the origin.

```
goto(x * 200, y * 200)
dot(10, "black" if inside else "red")
update() # display recent drawing in the window
```

(which avoids having to put the pen down and back up again, making the drawing faster).

In the example session above I have also drawn a 'progress bar' at the bottom of the window in green. It should not be difficult to figure out how to do this. (Hint: I just drew 1000 dots of size 5 evenly spaced between (-200, -225) and (200, -225).)

2.4 Estimating π

Each point lies within the square $(-1, -1)$ $(1, 1)$. The area of that square is 4 units. The unit circle has radius $r = 1$ and therefore area $\pi r^2 = \pi$. If the points are perfectly random then we would expect $\pi/4$ of them to lie within the circle.

You counted a **tally** of how many points lay within the circle experimentally. You also know the total **count** of points that you tested during the experiment. Knowing that $\pi/4$ of them ought to lie within the circle allows you to easily convert **tally** into an estimate of π .

2.5 Program development

Start with **count** set to 1000 and increase it to check that your estimate is getting closer to the actual value of π . Plotting all the points when **count** is large will slow down the simulation too much; instead plot each $(\text{count}/1000)^{\text{th}}$ point (in other words, plot 1000 points in total, sampled evenly over the whole experiment).

3 Selection sort [5 points]

Sorting and searching are two of the most important algorithms in computer science and programming. Selection sort is perhaps the simplest sorting algorithm and yet it is the fastest algorithm for very small collections (fewer than 8 or 9 elements). Some of the fastest sorting algorithms known make use of selection sort in some way. For example, Quick Sort is a 'divide-and-conquer' algorithm that solves a big sorting problem by breaking it into smaller and smaller sub-problems. When a sub-problem becomes small enough (involves fewer than 8 or 9 elements) then selection sort is often used to 'finish' the job of sorting these smallest sub-problems. If you have a good understanding of selection sort you will easily understand several other simple sorting algorithms. If you understand the (slightly) more complex Quick Sort algorithm, you will be able to use selection sort on the smallest sub-problems and implement a very efficient algorithm overall.

3.1 Task

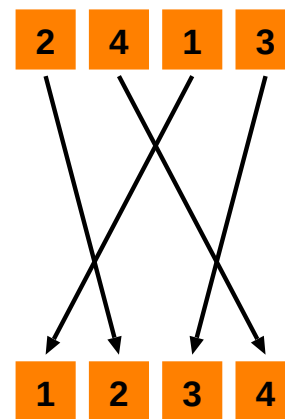
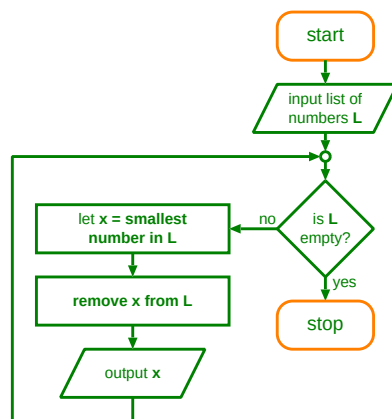
- create a function that produces an N-element list of random numbers
- create a function that sorts a list into ascending order
- create lots of lists, sort them, and verify that the answer is correct.

3.2 Example session

```
$ python3 selsort.py
[9, 1, 9, 5, 2, 5, 5, 7, 4, 5] [1, 2, 4, 5, 5, 5, 5, 7, 9, 9]
[7, 2, 6, 2, 3, 4, 1, 5, 5, 2] [1, 2, 2, 2, 3, 4, 5, 5, 6, 7]
[5, 0, 2, 2, 0, 2, 9, 5, 1, 2] [0, 0, 1, 2, 2, 2, 2, 5, 5, 9]
[1, 9, 6, 1, 6, 4, 5, 5, 7, 0] [0, 1, 1, 4, 5, 5, 6, 6, 7, 9]
[5, 1, 9, 7, 0, 0, 5, 9, 3, 0] [0, 0, 0, 1, 3, 5, 5, 7, 9, 9]
[3, 6, 6, 3, 6, 0, 6, 4, 1, 2] [0, 1, 2, 3, 3, 4, 6, 6, 6, 6]
[0, 5, 5, 0, 5, 8, 9, 0, 7, 3] [0, 0, 0, 3, 5, 5, 5, 7, 8, 9]
[8, 4, 0, 9, 2, 1, 2, 5, 9, 0] [0, 0, 1, 2, 2, 4, 5, 8, 9, 9]
[1, 1, 4, 3, 4, 7, 6, 1, 1, 8] [1, 1, 1, 1, 3, 4, 4, 6, 7, 8]
[6, 9, 1, 7, 9, 0, 9, 0, 2, 9] [0, 0, 1, 2, 6, 7, 9, 9, 9, 9]
[4, 8, 3, 4, 6, 1, 0, 0, 2, 7] [0, 0, 1, 2, 3, 4, 4, 6, 7, 8]
[1, 6, 5, 5, 7, 9, 0, 5, 6, 8] [0, 1, 5, 5, 5, 6, 6, 7, 8, 9]
[7, 0, 0, 8, 0, 9, 1, 2, 8, 8] [0, 0, 0, 1, 2, 7, 8, 8, 8, 9]
[5, 8, 7, 3, 4, 9, 1, 4, 4, 6] [1, 3, 4, 4, 4, 5, 6, 7, 8, 9]
[3, 6, 8, 0, 9, 7, 5, 6, 8, 0] [0, 0, 3, 5, 6, 6, 7, 8, 8, 9]
[7, 0, 3, 8, 4, 9, 9, 5, 9, 1] [0, 1, 3, 4, 5, 7, 8, 9, 9, 9]
[7, 0, 2, 2, 7, 7, 6, 5, 2, 0] [0, 0, 2, 2, 2, 5, 6, 7, 7, 7]
[5, 0, 1, 1, 4, 5, 0, 5, 2, 1] [0, 0, 1, 1, 1, 2, 4, 5, 5, 5]
[8, 5, 4, 0, 7, 3, 0, 9, 0, 6] [0, 0, 0, 3, 4, 5, 6, 7, 8, 9]
[7, 0, 9, 2, 9, 3, 5, 2, 4, 7] [0, 2, 2, 3, 4, 5, 7, 7, 9, 9]
```

3.3 An obvious sorting algorithm

At the beginning of this course we discussed an algorithm for sorting numbers into ascending order.



This algorithm is easy to implement in Python using lists. However, it creates a new list containing the sorted values which is not always what we want. Let's modify the algorithm slightly so that it sorts the list *in-place*, which means that the original list is rearranged to put the elements in order without creating a new list.

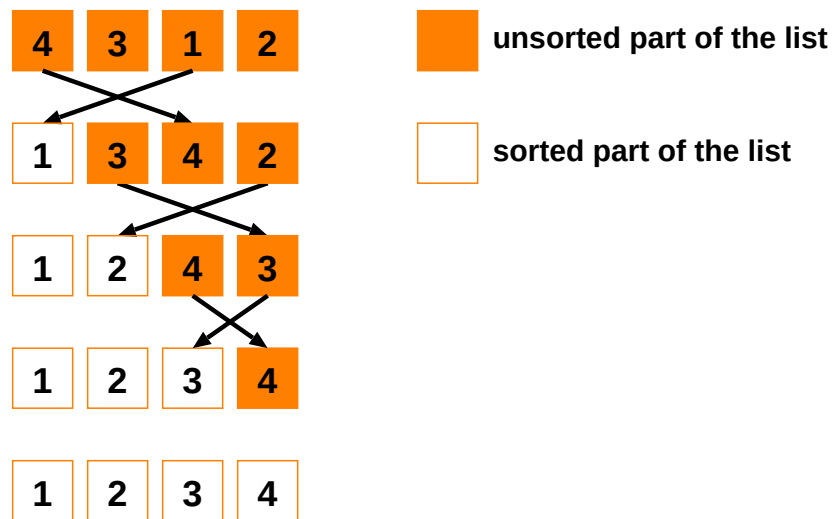
3.4 Selection sort

The algorithm is almost the same, except that we don't output each value in ascending order as we find it. Instead we move it to its correct place in the original list. If you think about the final list, the elements will be arranged in ascending order from left-to-right.

Consider a list with N elements, with indexes 0 to $N - 1$. To sort the list from indexes 0 to $N - 1$ the algorithm should start by finding the smallest element S . Let's say that element is at index I . The element S should really be at index 0. However, position 0 is not empty. If we are going to put the smallest element there, the element that is currently in position 0 has to go somewhere else. Where should we put it? It does not really matter, and conveniently we have just created an empty space at index I where the smallest element S used to be. We can therefore just swap the elements at positions I and 0, after which we know the first element is in the correct position.

Since we know the first element is now correct, we never need to look at it again. To find the second smallest element and place it in the correct location we can therefore just repeat the above algorithm but over the range 1 to $N - 1$. After that the element in position 1 will be correct.

The algorithm can then be repeated from position 2 to $N - 1$, then from 3 to $N - 1$, and so on. Finally we consider the range $N - 1$ to $N - 1$ which contains only one element, so we can stop. The list is now completely sorted.



3.5 Program structure

You can get away with writing just two functions and a loop.

The first function you should write, `ranlist(n)`, creates a list of `n` random numbers between 0 and `n`.

```
>>> ranlist(10)
[9, 1, 9, 5, 2, 5, 5, 7, 4, 5]
```

You can use the existing function `random.randint(minval, maxval)` to obtain a random number in the range `minval` to `maxval`, inclusive.

The second function you should write, `sel sort(l)`, sorts the list `l` in-place using the selection sort algorithm described in the previous section.

When you have these two functions, you can test your implementation of selection sort using the following loop:

```
for _ in range(20):  
    l = ranlist(10)  
    print(l, end="\t")  
    selectionSort(l)  
    print(l)  
    assert(l == sorted(l))
```

Note the use of `assert(predicate)` at the end of the loop. The `assert` function will stop your program immediately if it is called with a `predicate` argument that is `False`. In the loop above the assertion that is being made is that the list `l` is sorted by comparing it with the result of Python's own built-in `sorted()` function.